

Informe de entrega — Proyecto final

Proyecto: Registros clínicos — vista de tarjetas derivada de datos **Autor:** ronirck **Fecha:** 30 de agosto de 2026 **Aplicación en producción:** <https://proyecto-final-neon-five.vercel.app/> **Repositorio:** <https://github.com/ronirck/proyecto-final>

1. La aplicación

La aplicación es una página web interactiva que presenta un conjunto de registros clínicos como una rejilla de tarjetas navegable. El dominio es un extracto de historias clínicas: cada registro identifica a un paciente con un código de prefijo `HC-` y contiene su nombre completo, edad, especialidad médica, sede de atención escrita como «ciudad, país», fecha de la última visita, presión sistólica, un indicador de seguro activo y un campo de notas de texto libre. Son diez registros, de `HC-1001` a `HC-1010`, elegidos a propósito para contener nulos, cadenas vacías, tipos mezclados y campos agrupados, de modo que la capa de normalización tuviera un problema real que resolver.

En pantalla aparece primero una fila de métricas resumen —número de registros, edad promedio con su rango, presión sistólica media con su rango, porcentaje de pacientes con seguro y visitas dentro de una ventana móvil de treinta días— y debajo un bloque de controles: una barra de búsqueda y tres grupos de filtros, con especialidad y sede destacados y el filtro de seguro en una fila secundaria. Cada tarjeta muestra el nombre como título, el código y la edad como subtítulo, especialidad, sede y seguro como etiquetas de color, la presión y la última visita como datos secundarios y las notas al pie. Un contador indica cuántas tarjetas se muestran sobre el total, y cuando una combinación de filtros no deja resultados aparece un estado vacío con un botón para limpiarlos.

El stack es Next.js 15 con App Router, React 19 y TypeScript, desplegado en Vercel. El almacén en producción es Postgres 18 en Neon, y la conexión usa `@neondatabase/serverless`, única dependencia de tiempo de ejecución. La página raíz es un Server Component dinámico: consulta la tabla en cada petición, pasa las filas por la normalización y entrega datos ya limpios a un Client Component que resuelve filtrado y búsqueda en memoria. El sistema visual —paleta de acento, tipografía, espaciado, foco y comportamiento responsivo— está definido a mano en una hoja de estilos propia, sin framework de utilidades ni librería de componentes.

2. Herramientas utilizadas

El agente principal fue Claude Code, operado con la suscripción de Anthropic; toda la generación de código, la orquestación de subagentes, la ejecución de comandos y la redacción de documentación se hizo desde esa única herramienta. Para la base de datos se usó el servidor MCP de Neon, con el que se creó el proyecto Postgres, se obtuvo la cadena de conexión, se creó la tabla, se cargaron los diez registros, se perfiló la tabla viva y se ejecutaron las pruebas de datos hostiles. El despliegue se hizo en Vercel, importando el repositorio de GitHub y configurando la variable de entorno en su panel.

Debe quedar constancia explícita de lo que no se usó. No se recurrió a OpenRouter, ni a Google AI Studio, ni a Gemini, ni a Lovable, ni a Google Stitch, ni a ninguna otra pasarela o estudio de modelos de terceros, y no se configuró ninguna clave de API de terceros: la única credencial del proyecto es la cadena de conexión de Neon, y vive solo en variables de entorno. Tampoco se empleó ninguna herramienta externa de diseño —ni Figma, ni un generador de interfaces, ni plantillas compradas—: la dirección visual, la paleta clínica y todo el CSS los generó el propio agente a partir de la skill y del ajuste pedido en el comando.

3. Cumplimiento de las normativas

Norma 1 — Cero código manual. Todos los archivos de código los emitió el agente o uno de sus subagentes: la configuración del proyecto (`package.json`, `tsconfig.json`, `next.config.mjs`), el acceso a datos (`lib/db.ts`), la vista completa (`app/page.tsx`, `app/layout.tsx`, los tres componentes de `components/`), la hoja de estilos (`app/globals.css`), el esquema SQL (`db/schema.sql`), el cargador (`db/load.mjs`) y las sentencias de inserción (`db/seed.inserts.sql`). El único punto que habría tentado a una edición rápida —un fallo de validación de fechas en el cargador— no se tocó a mano: se delegó su diagnóstico y corrección al subagente. La cadena de conexión no se pega en el código; el agente generó `.env.local`, `.env.example` y `db/.env.local.example` con las instrucciones para rellenarla.

Norma 2 — La interfaz se deriva de los datos. El proyecto tiene una doble inyección de contexto. `data/seed.json` es el contrato del esquema: el documento que el agente leyó campo por campo para producir el perfil que gobierna todo lo demás, y la semilla desde la que se cargó la tabla; permanece en el repositorio con sus diez

registros como evidencia de que el diseño se dedujo de datos estructurados. La tabla `registros_clinicos` de Neon es la fuente en tiempo de ejecución: `lib/db.ts` hace `SELECT ... FROM registros_clinicos ORDER BY id` en cada petición y la aplicación nunca lee el JSON en producción. El esquema SQL se derivó del perfil, no al revés, y cada decisión de tipo en `db/schema.sql` lleva un comentario que la ata a una anomalía del dato real. El perfil detectó dos campos agrupados: `nombre_completo` se separa en `nombre_pila` (primer token) y `apellidos` (resto), y `sede` en `sede_ciudad` y `sede_pais`; en ambos casos se conserva el valor original y la separación ocurre una sola vez, en la normalización. Las coacciones de tipo se verificaron fila a fila tras la carga: HC-1002 traía la edad como "52" y quedó como 52; HC-1008, la presión como "148" y quedó como 148; HC-1005, el seguro como "SI" y quedó verdadero; HC-1006 no traía seguro y quedó nulo, no falso; HC-1003 traía las notas como cadena vacía y quedó nulo. La comprobación posterior confirmó diez filas, con dos nulos en edad, dos en presión, dos en seguro y tres en notas, y tipos de columna `integer`, `date` y `boolean` donde correspondía.

Norma 3 — Una skill y un comando propios. La skill `datos-a-interfaz` y el comando `/renderizar_tarjetas` existen como archivos versionados en `.claude/skills/` y `.claude/commands/`. La skill impone un orden fijo —primero se entiende el dato y después se dibuja— con cuatro pasos: perfilar la fuente, derivar los componentes del perfil, blindar contra nulos con una capa de normalización única y verificar contra una lista de comprobación. El comando encapsula ese flujo en seis pasos numerados que no se pueden saltar y deja un hueco para el ajuste que se le pase. En esta entrega se invocó con «destacar especialidad y sede como filtros, la vista debe consultar la tabla `registros_clinicos` en Neon, no leer el seed», y ambas condiciones se incorporaron sin romper los pasos anteriores.

Norma 4 — Dos agentes especializados. La separación de responsabilidades no es cosmética. `guardian-datos` es el único que toca la normalización; su configuración se lo dice: «nadie más toca la normalización; los componentes reciben datos ya limpios y confían ciegamente en tu trabajo». `depurador-web` tiene prohibido proponer una corrección antes de explicar la causa raíz. La delegación real entre ambos quedó en la bitácora: `guardian-datos` recibió tres encargos —perfilar el seed y derivar esquema y cargador; dejar el esquema idempotente y generar las inserciones; y producir la capa `lib/normalizar.ts` con un contrato de tipos exacto y sin dependencias—, y en el tercero se le indicó que el orquestador escribiría los componentes de React y que él no debía tocarlos. Devolvió la capa con cuarenta y seis comprobaciones de datos hostiles, todas superadas. Cuando después `depurador-web` encontró el fallo de fechas, respetó la frontera: corrigió el cargador `db/load.mjs`, que es ruta de escritura, y dejó constancia de que `lib/normalizar.ts` era territorio de `guardian-datos` y solo lo usaría como referencia.

Norma 5 — Depuración delegada. Se pidió insertar en Neon un registro con `ultima_visita` en formato inválido, "31/02/2026", y `presion_sistolica` como texto no numérico, "no-medida". Postgres rechazó ambos valores de forma atómica —`date/time field value out of range: "31/02/2026"` e `invalid input syntax for type integer: "no-medida"`— y no se creó ninguna fila. Aparentemente el esquema tipado había hecho su trabajo y no quedaba nada que corregir, pero `depurador-web` siguió el protocolo y encontró un fallo latente que esa barrera ocultaba. La función `aFechaONull` de `db/load.mjs` tenía tres ramas y ninguna validaba el calendario: la rama ISO devolvía cualquier cadena con forma `YYYY-MM-DD` sin comprobar rangos; la rama `dd/mm/yyyy` solo rellenaba ceros y convertía "31/02/2026" en "2026-02-31" sin preguntarse si el 31 de febrero existe; y la rama de reserva con `Date.parse` hacía un desbordamiento silencioso hacia el 3 de marzo. La consecuencia es la que importa: si esa fecha apareciera en `data/seed.json`, `normalizarRegistro` la volvería "2026-02-31", se pasaría a la columna `DATE` y Postgres la rechazaría; y como el cargador corre toda la inserción en una única transacción con `ROLLBACK`, un solo registro con una fecha imposible habría abortado la carga completa de la tabla, no solo la fila defectuosa. La capa de lectura ya se protegía con una validación de ida y vuelta por `Date.UTC`; la de escritura no.

La corrección se limitó a `db/load.mjs`: un helper `armarFechaISO(y, mo, d)` que valida el rango de mes y día, construye la fecha con `new Date(Date.UTC(...))` y comprueba que los componentes de vuelta coincidan; si el día civil no existe, devuelve nulo. Las tres ramas de `aFechaONull` pasan ahora por él y ante un resultado nulo emiten el aviso ya existente en lugar de una fecha ISO inválida. La verificación neutralizó "31/02/2026", "2026-13-01", "31-02-2026" y "2026-02-29" —febrero no bisiesto—, conservó "2028-02-29", "14/02/2026" y "2026-08-21", y volvió a normalizar los diez registros del seed sin un solo aviso. `depurador-web` señaló tres fallos latentes que decidió no parchear —que `aEnteroONull` no acota rangos clínicos, que `Date.parse` sigue siendo dependiente del motor para formatos exóticos y que `separarSede` convertiría un número en texto sin avisar— porque son decisiones de esquema que corresponden a `guardian-datos`, no defectos de la ruta de escritura, y parchearlos habría mezclado una corrección acotada con cambios de alcance sin evaluar. Se pide la causa raíz antes que el parche porque un parche aplicado sin entender el fallo tapa el síntoma y reaparece en producción, y además impide distinguir el fallo que hay que arreglar de los que solo hay que anotar.

Norma 6 — Despliegue público. El proyecto está en <https://proyecto-final-neon-five.vercel.app/>, con `DATABASE_URL` configurada en el panel de entornos de Vercel y no en el código. Antes de guiar el despliegue se simuló el build con `npm ci` y `next build` y el archivo `.env.local` retirado, y terminó con éxito y sin errores de tipos: la página raíz es dinámica y solo necesita la conexión en tiempo de petición, así que un primer despliegue sin la variable no rompe el build. Contra la URL de producción se comprobó respuesta `HTTP 200` servida por Vercel con cabeceras dinámicas, las diez tarjetas renderizadas desde la tabla, el contador «Mostrando 10 de 10 registros», la ausencia total de `undefined`, `null`, `NaN` u `[object Object]` en el DOM, y métricas idénticas a las locales.

4. Diseño de la orquestación

La skill `datos-a-interfaz` es un procedimiento, no una plantilla. Obliga a leer la fuente real y a producir una tabla de perfil —nombre de cada campo, tipo predominante, porcentaje de nulos, cardinalidad, si es agrupado y rango si es número o fecha— y a usarla como contrato. De ahí deduce los componentes: baja cardinalidad se vuelve filtros combinables, texto libre una única búsqueda insensible a mayúsculas y acentos, fecha una ventana móvil calculada en tiempo de ejecución y nunca meses fijos, y número una métrica resumen. Impone además la capa de normalización única entre fuente y vista y exige una prueba de humo con datos hostiles antes de dar nada por bueno.

El comando `/renderizar_tarjetas` orquesta esa skill en seis pasos que el agente recorre en orden: perfilar y mostrar la tabla, delegar la normalización en `guardian-datos` y esperar su reporte, generar la vista sobre los datos limpios, aplicar el ajuste puntual, probar con datos hostiles y, por último, recorrer la lista de verificación y reportar el estado de cada punto, delegando en `depurador-web` cualquier fallo. Ese sexto paso, el de verificación explícita, es lo que separa al comando de una petición suelta al chat. Un prompt escrito en la conversación se ejecuta una vez, con el criterio del momento, y desaparece: si mañana hay que regenerar la vista, hay que recordar qué se pidió y con qué matices. Un prompt guardado como comando es un artefacto del repositorio que define el flujo, el orden, las delegaciones obligatorias y el criterio de aceptación, y produce el mismo procedimiento en otra máquina y con otra persona al mando. La reproducibilidad no está en el resultado sino en el proceso que lo genera, y ese proceso está versionado.

5. Manejo de valores ausentes

La decisión de fondo fue que los componentes nunca vean un valor ausente. Toda lectura pasa por la capa de normalización, que sustituye `null`, `undefined` y cadena vacía por una etiqueta de respaldo legible —«Sin nombre», «Sin especialidad», «Sin sede», «Sin dato», «Sin visita registrada» o un guion— y jamás por la cadena literal `"null"` ni por `NaN`. Cada campo con respaldo queda marcado en un mapa `esRespaldo`, y las filas cuyo valor sea de respaldo se excluyen del cálculo de métricas para no contaminar los promedios. Los campos de agrupación exponen además una clave estable, `"__sin_dato__"`, para que los filtros ofrezcan un grupo «Sin dato» propio en lugar de descartar esas filas.

El registro `HC-1010` es el caso extremo: solo tiene código y notas, y los otros siete campos son nulos. Con la decisión tomada, su tarjeta se renderiza con normalidad —título «Sin nombre», edad «Sin dato», etiquetas «Sin especialidad» y «Sin sede»—, cuenta en el total de registros pero no entra en la edad promedio, ni en la presión media, ni en el porcentaje de seguro, ni en el conteo de visitas recientes, y sigue siendo localizable por el texto de sus notas. Sin esa decisión, el resultado sería una de tres cosas: la tarjeta mostraría `undefined` o el texto `null` en cada hueco; el filtrado se caería al invocar un método de texto sobre un valor nulo; o la edad promedio se desplomaría al promediar sobre un cero inexistente.

El registro `HC-1008` ilustra un matiz distinto: su `nombre_completo` es `"Ricardo"`, un solo token, porque en el sistema de origen ese paciente no tiene apellido registrado, y así lo dicen sus notas. La normalización separa el nombre en `nombre_pila`, que queda como «Ricardo», y `apellidos`, que queda como el respaldo «—» con su marca de ausencia activada. Lo importante es que el apellido ausente no se inventa: no se duplica el nombre, no se rellena con una inicial, no se deja una cadena vacía que reviente más abajo. Es un dato real que falta, y se muestra como tal. Si la lógica de separación no distinguiera «un token» de «varios tokens», el `slice` del resto produciría una cadena vacía que la vista trataría como un apellido legítimo, y la búsqueda y el orden operarían sobre un valor sin sentido.

6. Prompts utilizados

Los prompts se dividen en dos categorías. Los **estructurados** están guardados como archivos de configuración del agente y se aplican en cada sesión sin reescribirlos: `CLAUDE.md` fija el contexto y las seis normas; `.claude/skills/datos-a-interfaz/SKILL.md` define el flujo de construcción de vistas; `.claude/commands/renderizar_tarjetas.md` es un prompt guardado con los seis pasos descritos en la sección 4; y `.claude/agents/guardian-datos.md` y `.claude/agents/depurador-web.md` son los prompts de sistema de los dos

subagentes. Los **conversacionales** se escribieron en el chat, se ejecutaron una vez y quedaron en la bitácora; se transcriben a continuación por fase.

Configuración del entorno. Esta fase no tuvo prompt conversacional propio: el entorno lo fijaban los archivos estructurados y la conexión previa del servidor MCP de Neon, cuya ausencia se había diagnosticado y resuelto antes según la bitácora («la instrucción asume un servidor MCP de Neon disponible... decisión del usuario: añadir el MCP de Neon»).

Capa de datos.

Usa el agente guardian-datos para: perfilar data/seed.json, derivar el esquema SQL a partir de ese perfil, crear la tabla en Neon vía MCP y cargar los 10 registros. Las credenciales van en .env.local y .env.local debe estar en .gitignore. Nunca escribas la cadena de conexión en el código.

Generación de interfaz.

/renderizar_tarjetas destacar especialidad y sede como filtros, la vista debe consultar la tabla registros_clinicos en Neon, no leer el seed

Dentro de ese flujo, la delegación estructurada a guardian-datos pedía «generar lib/normalizar.ts (TS, ESM, sin dependencias) con RegistroCrudo, RegistroNormalizado (contrato exacto), normalizarTexto, normalizarRegistro, normalizarRegistros, calcularMetricas... El orquestador escribe los componentes React; el agente NO los toca».

Depuración.

Inserta en Neon vía MCP un registro con ultima_visita en formato inválido (por ejemplo "31/02/2026") y presion_sistolica como texto no numérico. Luego invoca a depurador-web para que diagnostique la causa raíz y aplique la corrección. Registra el reporte completo en BITACORA.md y borra la fila de prueba al terminar.

Despliegue.

El repo ya está en GitHub. Prepara el despliegue en Vercel y guíame para conectarlo. DATABASE_URL va como Environment Variable en el panel de Vercel, con el mismo valor de .env.local. Si el build falla, invoca a depurador-web para diagnosticar la causa raíz antes de corregir.

<https://proyecto-final-neon-five.vercel.app/> este fue el url de productivo

7. Documentos de apoyo

El repositorio conserva los artefactos que sirven de evidencia del proceso. BITACORA.md es el registro cronológico de cada prompt relevante, cada delegación entre agentes y cada error diagnosticado con su causa raíz; contiene las tablas de verificación fila a fila de la carga, el reporte de la prueba de datos malformados y la verificación del despliegue. db/PERFIL.md es el perfil del esquema campo por campo sobre el dato real, con presencia y ausencia contando por separado null y cadena vacía, tipos crudos, cardinalidad, rango y las contradicciones entre el dato real y lo que el proyecto asumía. db/schema.sql es el esquema SQL derivado de ese perfil, con cada decisión de tipo comentada. CLAUDE.md fija el contexto y las seis normas del repositorio. Los archivos de configuración del agente — .claude/skills/datos-a-interfaz/SKILL.md, .claude/commands/renderizar_tarjetas.md, .claude/agents/guardian-datos.md y .claude/agents/depurador-web.md — son los prompts estructurados que definen el flujo, el comando que lo orquesta y los dos subagentes. Como apoyo, README.md resume la arquitectura y los pasos de despliegue, y db/seed.inserts.sql conserva las diez sentencias de carga idempotentes generadas a partir del seed.